

MEDHAVA

Build queue

8 tasks, in order, each an independently verifiable vertical slice.

Every task carries the fields §51 asks for, and one it does not: **why it is in this position**. An ordered list with no argument for the order is a list somebody has to take on trust, and the order is the part most worth disagreeing with.

No task carries an estimate in days. There is no basis for one here — no velocity from this project and no comparable delivered — and inventing "three days" would be the same class of statement as the two figures reported wrong this session. Complexity is carried instead, as S/M/L with the reason it is that size.

The order at a glance

ID	Task	Risk	Size	After	Blocked
Q01	Put the sixteen browser apps inside the gated suite	LOW	S	—	—
Q02	Purchase order to goods receipt, on the real database	MEDIUM	L	Q01	—
Q03	One working day, end to end, as a single test	MEDIUM	M	Q02	—
Q04	A first deployment, and a smoke test that proves it answered	HIGH	M	Q03	yes
Q05	Write down who the attacker is, then test as them	HIGH	M	Q01	—
Q06	A shift rota, and attendance measured against it	LOW	M	Q01	—
Q07	A query engine behind the report builder	MEDIUM	L	Q02	—
Q08	Behaviour at a volume nobody has tried	MEDIUM	M	Q02, Q07	—

Q01 · Put the sixteen browser apps inside the gated suite

Regression safety for work already done

Risk	LOW — Adds a check to an existing pipeline; changes no product behaviour. The realistic bad outcome is that it goes red on day one, which is the point.
Complexity	S — One script already exists (build_deep.js, check_deep.js) and already passes when run by hand. The work is wiring and whatever it turns out those two find once they must pass every time.
Requirements	CAP-CI
Depends on	nothing
Files expected to change	package.json , brand/site/registry.js , docs/verification/EVIDENCE.md , brand/suite/deep/apps.js , brand/suite/deep/build_deep.js , brand/suite/deep/check_deep.js , .github/workflows/ci.yml

Why here in the order. Sixteen apps are at score 3 solely because their tests are not gated. This is the only task in the queue that raises real scores without writing a feature, and it protects everything built after it. Left undone, every later slice is built on sixteen things nothing is watching.

Acceptance criteria.

- `npm test` runs build_deep.js and check_deep.js and fails when either does
- a deliberately broken control in one app turns the suite red, proven before green
- the run is recorded through tools/evidence.js at exit 0
- registry.js raises those apps only after that recorded run exists

Test plan. The existing per-app self-tests and the click-through audit, unchanged — the work is making them mandatory, not writing new ones.

Verification plan. checkregistry.js refuses the raised rung until the command appears in EVIDENCE.md at exit 0, so this cannot be marked done by editing a register.

Evidence required. A V-APPS entry in docs/verification/EVIDENCE.md.

Q02 · Purchase order to goods receipt, on the real database

Buying — the first half of the working day that does not exist

Risk	MEDIUM — It writes to stock, which is the number every other module reads. A receipt that posts twice or posts outside its company is the most expensive defect available in this system.
Complexity	L — A new server module, two screens, the vendor record behind them, and the ledger posting on receipt. Comparable to the sales slice, which was the largest piece of work done so far.
Requirements	APP-07-01 , APP-07-02 , APP-03-01
Depends on	Q01
Files expected to change	medhava/server/purchase.js , medhava/test/purchase.test.js , medhava/web/index.html , core/schema.postgres.sql , brand/site/registry.js

Why here in the order. Stock and sales exist; nothing puts stock there. Until something does, every demonstration begins with a seeded quantity nobody bought, which is the shape of a demo rather than a system.

Acceptance criteria.

- a purchase order raised against a vendor, received in part, and received in full
- receiving increases stock at the receiving location and nowhere else
- the ledger entry balances and carries the company it belongs to
- a receipt against another company's order is refused by the database, not the code
- a partial receipt leaves the order open with the right outstanding quantity

Test plan. medhava/test/purchase.test.js, each rule proven red before green, driven through a non-superuser role so the policies are live.

Verification plan. `npm run medhava` in the gated suite, recorded through evidence.js.

Evidence required. A recorded run, and the registry row for Procurement raised by it.

Q03 · One working day, end to end, as a single test

The thing that decides whether this is level 4

Risk	MEDIUM — It will find the seams between modules that per-module tests cannot, which is what it is for. Expect it to fail first for real reasons.
Complexity	M — No new feature — one test that drives what Q02 and the sales slice already built, plus whatever it exposes between them.
Requirements	APP-03-01 , APP-05-01 , APP-07-01
Depends on	Q02
Files expected to change	medhava/test/day.test.js , brand/site/audit.js

Why here in the order. The maturity level in this register names this exact test as the condition for level 4. Writing the condition down and never running it is how a level becomes a mood.

Acceptance criteria.

- buy, receive, sell, and see all of it in the ledger, in one test, one company
- stock returns to its opening figure when the same quantity is bought and sold
- the same run against a second company sees none of the first company's rows

Test plan. medhava/test/day.test.js — a scenario, not a unit test.

Verification plan. In `npm run medhava` , recorded. The maturity level in audit.js may only be raised to 4 in the same commit that adds this passing.

Evidence required. A recorded run, cited by the maturity level itself.

Q04 · A first deployment, and a smoke test that proves it answered

CAP-DEPLOY — nothing has ever been installed anywhere

Risk	HIGH — Everything about production is untested here: no server has run this, no certificate has been issued, no backup has been restored. The runbook has never been followed by anybody.
Complexity	M — The runbook, the nginx blocks and the systemd unit are written. The work is doing it once for real and fixing what the writing got wrong.
Requirements	CAP-DEPLOY , CAP-MONITOR
Depends on	Q03
Files expected to change	DEPLOYMENT.md , deploy/ , brand/site/registry.js

Why here in the order. Deployment blocks monitoring, blocks any production-readiness claim, and is the single largest gap between what this project describes and what it is. It is placed after Q03 because deploying something that cannot do a day's work is a demonstration.

Acceptance criteria.

- the product runs on a machine that is not a developer's
- a smoke test hits the running instance and asserts a real response, from outside it
- the database role there is neither superuser nor table owner — checked on that box
- a restore from backup is performed once and the restored data checked

Test plan. A smoke test run against the deployed URL, not against localhost.

Verification plan. Recorded through evidence.js from a machine that can reach it.

Evidence required. A recorded run naming the host, and the registry row raised only then.

Blocked.

- Needs a server, a domain and credentials, none of which exist and none of which this repository may ever hold. This task cannot start inside this environment: the egress proxy refuses everything but package registries and GitHub.

Q05 · Write down who the attacker is, then test as them

CAP-THREATMODEL — isolation is proven against an honest client only

Risk	HIGH — Not that the work is hard, but that its findings will be. A first adversarial pass against a system nobody has attacked usually finds something.
Complexity	M — A threat model document from the registers that already exist, and a test file that tries the things it names.
Requirements	CAP-THREATMODEL , CAP-ISOLATION
Depends on	Q01
Files expected to change	THREAT_MODEL.md , medhava/test/adversarial.test.js

Why here in the order. Every isolation test here drives the system the way it is meant to be driven. That proves the policy is switched on; it does not prove it cannot be got around. The strongest claim in this repository currently rests on tests that never try.

Acceptance criteria.

- a written threat model naming who, what they want, and what they can already reach

- a test that attempts a cross-company read by every route the API exposes
- a test that confirms the app role cannot disable a policy, and fails if it can
- every attempt is refused, and each refusal names the rule that refused it

Test plan. medhava/test/adversarial.test.js.

Verification plan. In the gated suite, recorded. Red first: temporarily grant the app role ownership and confirm the suite goes red, then take it back.

Evidence required. A recorded run, plus the red-first result written into the commit.

Q06 · A shift rota, and attendance measured against it

The clearest NO APP finding against a competitor's named product

Risk	LOW — Self-contained. It reads the roster and writes a schedule; nothing downstream depends on it yet.
Complexity	M — A new record type, a screen to build the rota on, and the join to attendance that gives it its point.
Requirements	APP-16-01
Depends on	Q01
Files expected to change	medhava/server/roster.js , medhava/test/roster.test.js , core/schema.postgres.sql

Why here in the order. Attendance is captured and there is no schedule to measure it against, so "late" and "absent" have no definition in the system. It is the one NO APP row that is already half-built by something else here.

Acceptance criteria.

- a rota published for a week, per person, per unit
- attendance compared to the rota produces late, absent and unplanned-present
- changing a published rota keeps the old one, effective-dated, and never overwrites

Test plan. Per-rule tests, red before green.

Verification plan. In the gated suite, recorded.

Evidence required. A recorded run of the gated suite, and the Staff & Contractors row raised only by it. Until then this is the largest NO APP row still open.

Q07 · A query engine behind the report builder

CAP-ANALYTICS — the dashboards compute over an in-page store

Risk	MEDIUM — A reporting layer that can read across companies is the fastest way to defeat the isolation everything else here rests on. It must run as the same restricted role as everything else, with no exception.
Complexity	L — A query builder, its safety boundary, and rewiring two existing screens onto it.
Requirements	CAP-ANALYTICS , APP-21-01 , APP-21-02
Depends on	Q02
Files expected to change	medhava/server/reports.js , medhava/test/reports.test.js

Why here in the order. The dashboards are the most convincing thing in the repository to look at and the least connected to anything. Every figure on them today is computed from data the page itself holds.

Acceptance criteria.

- a report defined once and run against the real database
- the same report run by two companies returns each company's own figures only
- no report can be written that reads a row its runner could not read directly
- a group figure is the sum minus inter-company trade, as core.test.js already proves

Test plan. Including an attempt to write a cross-company report, which must be refused.

Verification plan. In the gated suite, recorded.

Evidence required. A recorded run that includes the refused cross-company report, because the refusal is the part worth having on record.

Q08 · Behaviour at a volume nobody has tried

CAP-SCALE — every test here runs on a handful of rows

Risk	MEDIUM — The likely finding is that some query is fine at ten rows and not at production volume. Better found here than by a business at month end.
Complexity	M — A generator for realistic volume, and the measurements. No feature.
Requirements	CAP-SCALE
Depends on	Q02 , Q07
Files expected to change	medhava/test/volume.test.js

Why here in the order. Nothing in this repository supports any statement about speed, and the schema has 151 tables with row-level security on all of them — which is exactly the shape where a missing index is invisible until it is not.

Acceptance criteria.

- a stated volume per table that a real business of this size would reach in a year
- the working-day scenario run at that volume, with times recorded
- every query plan for the slowest ten reviewed, and each index added named

Test plan. A load scenario, not part of `npm test` — it is too slow — but recorded.

Verification plan. Recorded through evidence.js with the volumes in the entry.

Evidence required. A recorded run carrying the numbers, so a later claim about speed has something to disagree with.

What is deliberately not in this queue

The 98 apps that are specified and not built are not listed here as 98 tasks. A queue that long is a backlog, and a backlog is not an order — it is a place things go to stop being decided about. These eight are what the measurements in the other audit documents actually point at: the regression hole, the missing half of a working day, the deployment that blocks the whole top of the scale, and the two claims — isolation and speed — that rest on tests which have never tried to break anything.

Where every number here comes from

Nothing in this document is typed. It is generated by `node brand/delivery/website/mkaudit.js` and every figure is read at that moment from the register that owns it:

Fact	Register	Gate
modules and apps	<code>brand/site/modules.js</code>	<code>checkneutral.js</code> , <code>checkshape.js</code>
what each has reached	<code>brand/site/registry.js</code>	<code>checkregistry.js</code>
the 0–5 score and the queue	<code>brand/site/audit.js</code>	<code>checkaudit.js</code>
the capability comparison	<code>brand/site/zoho.js</code>	<code>checkzoho.js</code>
rules and their proofs	<code>brand/site/rules.js</code>	<code>checkrules.js</code>
recorded runs	<code>docs/verification/EVIDENCE.md</code>	<code>tools/evidence.js --check</code>

A figure in this document that disagrees with its register means the document is stale. Regenerate it; `npm test` refuses a stale one.

Every technical word above, in plain language

13 words. Every technical term this document uses, in plain language, with an everyday comparison. Nothing here assumes you already know any of them.

module

One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together.

Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.

database

Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.

Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.

table

One kind of information inside the database — all your customers in one, all your orders in another.

Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.

row

One single record — one customer, one order, one payment.

Register mein ek line. Ek line matlab ek entry.

schema

The written plan of what information the system keeps and how the pieces connect.

Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.

row-level security

A lock inside the database itself, so one business physically cannot read another business's records — even if the software above it has a bug.

Taala darwaze pe nahin, tijori pe. Guard so bhi jaaye toh bhi tijori band rehti hai.

backup

A copy of everything, kept somewhere else, so a mistake or a failure does not lose your work.

Zaroori kaagzaat ki photocopy, doosri jagah rakhi hui. Asli jal jaaye toh bhi kaam nahin rukta.

API

The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer.

Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.

queue

A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report.

Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.

environment

A separate running copy of the system — one for trying things, one that customers actually use.

Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho.

deployment

Putting a new version of the software in place so people start using it.

Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.

model

The piece of artificial intelligence that reads or writes text, tags a photograph, or answers a question.

Ek bahut padha-likha assistant. Kaam accha karta hai, par har baat pe usse poochho toh kharcha aur waqt dono lagta hai.

role

What a person is allowed to see and do — a manager sees more than a counter staff member.

Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.

Medhava · One business operating system · Any industry